Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică
Departamentul Ingineria Software și Automatică

Laboratory Work 5:

Track A: The File System with Your Own Hands (Linux)

Elaborated: gr. FAF-243
Poiata Calin

Verified:
Patricia Reițman

Chișinău – 2026

Table of Contents

Environment & Toolchain 1
Purpose of the Laboratory Work 1
Part 1. The Tree and File Types 1
1. Inspecting Root and File Types 1
2. Observe Answers for Part 1 2
Part 2. Inodes and Links 2
1. Inodes, Hard Links, and Symbolic Links 2
2. Observe Answers for Part 2 3
Part 3. Permissions 4
1. File Permissions, Umask, and Directory Traversal 4
2. Observe Answers for Part 3 5
Part 4. Disks and the Kernel as Files 6
1. Storage Disks, Partition Geometry, and `/proc` 6
2. Observe Answers for Part 4 7
Stretch Goals 7
1. Magic Numbers 7
2. Deleting an Open File 8
Conclusion 8

Environment & Toolchain

- **Host Operating System:** Linux (Arch Linux x86_64, Kernel 7.2.9-arch1-1)
- **Host Shell:** GNU bash, version 5.3.20(1)-release (`x86_64-pc-linux-gnu`)
- **Filesystem Under Test:** Native Linux ext4, tmpfs, and procfs pseudo-filesystem
- **Core Introspection Utilities:** `stat` , `ls` (coreutils 9.5), `file` , `du` , `df` , `lsblk` , `truncate` , `xxd`
- **Document Processor:** Typst 0.15.1 (`typst`)

Purpose of the Laboratory Work

1. Investigate the hierarchical tree structure of the Linux filesystem, distinguishing file types (regular files, directories, character devices, block devices, and symbolic links) directly through inode metadata.
2. Demystify the separation between human-readable directory filenames and physical on-disk inodes.
3. Observe and document the lifecycle of hard links, understanding that deletion (`rm`) performs an `unlink()` operation that decrements reference counters rather than wiping physical storage blocks.
4. Contrast symbolic links (soft links) with hard links, demonstrating broken link behavior (dangling pointers) and quantifying why a symlink's byte size strictly matches its target path string length.
5. Analyze directory link counts, sparse files, and the critical distinction between apparent file size and physical disk block allocation.
6. Explore Linux Discretionary Access Control (DAC), verifying class precedence (Owner → Group → Others), umask calculation, directory traversal (`x`) versus enumeration (`r`), SetUID execution, and the sticky bit.
7. Inspect storage devices, partition tables, and kernel runtime state exposed via the `/proc` virtual filesystem.

Part 1. The Tree and File Types

1. Inspecting Root and File Types

Every Unix filesystem represents storage objects through a unified directory hierarchy rooted at `/` . The first letter in the file mode string produced by `ls -l` indicates the file's primary inode type:

- `-` : Regular file
- `d` : Directory
- `c` : Character special device (unbuffered byte-stream I/O)
- `b` : Block special device (buffered, block-oriented random-access I/O)
- `l` : Symbolic link (contains pathname of another file)

Running the inspection commands from the normal user session:

```
$ ls /
bin  dev  etc  home  lib  lib64  proc  run  sbin  sys  tmp  usr  var

$ ls -ld /etc /etc/passwd /bin /dev/null /tmp
drwxr-xr-x   8 root root 143360 Oct 27 15:15 /bin
crw-rw-rw-   1 root root   1, 3 Nov  2 13:30 /dev/null
drwxr-xr-x 130 root root  12288 Nov  2 14:26 /etc
-rw-r--r--   1 root root   3265 Oct  1 12:32 /etc/passwd
drwxrwxrwt  22 root root    540 Nov  2 14:38 /tmp
```

Inspecting device files under `/dev` and testing file signatures:

```
$ ls -l /dev | head -n 10
total 0
crw-r--r--   1 root root  10, 235 Nov  2 13:30 autofs
drwxr-xr-x   2 root root      140 Nov  2 13:31 block
crw-rw----   1 root root  10, 234 Nov  2 13:30 btrfs-control
drwxr-xr-x   3 root root       60 Nov  2 13:30 bus
drwxr-xr-x   2 root root     4640 Nov  2 14:25 char
crw-------   1 root root   5,   1 Nov  2 13:30 console
lrwxrwxrwx   1 root root       11 Nov  2 13:30 core -> /proc/kcore
drwxr-xr-x  10 root root      200 Nov  2 14:25 cpu
crw-------   1 root root  10, 259 Nov  2 13:30 cpu_dma_latency

$ file /usr/bin/ls /etc/passwd /bin
/usr/bin/ls: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=d6281af586ce362d9c1029002636397df0cf21f1, for GNU/Linux 4.4.0, stripped
/etc/passwd: ASCII text
/bin:        directory

$ du -sh /var/log 2>/dev/null ; du -sh /usr 2>/dev/null
3.2G  /var/log
25G /usr
```

2. Observe Answers for Part 1

1. Type of each path from the first letter of `ls -l`:

- `/etc` : `d` → Directory
- `/etc/passwd` : `-` → Regular file
- `/bin` : `d` → Directory (in classic setups `l` for symbolic link)
- `/dev/null` : `c` → Character special device
- `/tmp` : `d` → Directory (with sticky bit `t`)

2. What is `/bin` on your system: a directory or a link, and to where? On this Arch Linux system, `/bin` is a directory holding binary executables directly integrated with `/usr/bin`. On traditional Linux distributions implementing the UsrMerge specification, `/bin` is a symbolic link pointing to `usr/bin` (`/bin` → `usr/bin`).

3. Find one character device and one block device in `/dev`:

- **Character Device:** `/dev/null` (major 1, minor 3), `/dev/console` (major 5, minor 1), and `/dev/zero` (major 1, minor 5).
- **Block Device:** `/dev/nvme0n1` (major 259, minor 0) and its partition `/dev/nvme0n1p2`.

4. Which is bigger on your laptop, `/var/log` or `/usr`, and by roughly how much? `/usr` is **25 GB**, whereas `/var/log` is **3.2 GB**. `/usr` is significantly larger by roughly **21.8 GB** (approximately 7.8 times larger). This disparity is inherent: `/usr` stores system binaries, static shared libraries, dynamic modules, compiler toolchains, and application assets, while `/var/log` only stores runtime operational logs and systemd journal data.

Part 2. Inodes and Links

1. Inodes, Hard Links, and Symbolic Links

In Unix file systems, a file is not identified by its filename. The fundamental data structure is the **inode** (index node), which stores all metadata: file type, access mode, owner UID, group GID, byte size, timestamp records, link count, and block pointers. A filename is merely an entry in a directory table mapping a string to an inode number.

```
$ echo hello > notes.txt
$ stat notes.txt
  File: notes.txt
  Size: 6           Blocks: 8          IO Block: 4096   regular file
Device: 0,48  Inode: 1518        Links: 1
Access: (0666/-rw-rw-rw-)  Uid: ( 1000/   calin)   Gid: ( 1000/   calin)
Access: 2026-11-02 14:40:21.820346533 +0200
Modify: 2026-11-02 14:40:21.821478340 +0200
Change: 2026-11-02 14:40:21.821478340 +0200
 Birth: 2026-11-02 14:40:21.820346533 +0200

$ ls -li notes.txt
1518 -rw-rw-rw- 1 calin calin 6 Nov  2 14:40 notes.txt

$ ln notes.txt hard.txt
$ ls -li notes.txt hard.txt
1518 -rw-rw-rw- 2 calin calin 6 Nov  2 14:40 hard.txt
1518 -rw-rw-rw- 2 calin calin 6 Nov  2 14:40 notes.txt

$ rm notes.txt
$ cat hard.txt ; ls -li hard.txt
hello
1518 -rw-rw-rw- 1 calin calin 6 Nov  2 14:40 hard.txt
```

Testing symbolic link behavior and broken pointers:

```
$ ln -s hard.txt soft.txt
$ ls -li hard.txt soft.txt
1518 -rw-rw-rw- 1 calin calin 6 Nov  2 14:40 hard.txt
1519 lrwxrwxrwx 1 calin calin 8 Nov  2 14:40 soft.txt -> hard.txt
$ cat soft.txt
hello

$ rm hard.txt
$ cat soft.txt ; ls -l soft.txt
cat: soft.txt: No such file or directory
lrwxrwxrwx 1 calin calin 8 Nov  2 14:40 soft.txt -> hard.txt
```

Testing directory link counts and sparse files:

```
$ mkdir d ; ls -ld d
drwxrwxrwx 2 calin calin 40 Nov  2 14:40 d

$ mkdir d/x d/y ; ls -ld d
drwxrwxrwx 4 calin calin 80 Nov  2 14:40 d

$ echo x > one.txt ; ls -l one.txt ; du -h one.txt
-rw-rw-rw- 1 calin calin 2 Nov  2 14:40 one.txt
4.0K  one.txt

$ truncate -s 1G big.img ; ls -lh big.img ; du -h big.img
-rw-rw-rw- 1 calin calin 1.0G Nov  2 14:40 big.img
0 big.img
```

2. Observe Answers for Part 2

1. Which number in `stat` is the inode, and which tells how many names the file has?

- The inode number is displayed after `Inode:` (`1518`).
- The count of filenames pointing to this inode is displayed after `Links:` (`Links: 1` , and `Links: 2` after creating `hard.txt`).

2. After `rm notes.txt` , why could you still read `hard.txt` ? What does `rm` really remove? `notes.txt` and `hard.txt` are simply two directory table entries pointing to the exact same inode `1518` . The `rm` command invokes the `unlink()` system call, which deletes the named mapping from the directory table and decrements the inode's `nlink` field from 2 to 1. Because `nlink > 0` , the underlying inode and its data blocks remain intact on disk. The `rm` command

removes a **directory name**, not the file data itself. Storage blocks are only released when `nlink = 0` and all open file descriptors to the file are closed.

3. Why did `cat soft.txt` fail at the end, although `soft.txt` still exists? How big is `soft.txt` and why exactly that size? A symbolic link is a separate file with its own unique inode (`1519`) whose content is the text string specifying the path to the target (`"hard.txt"`). When `hard.txt` was unlinked, `soft.txt` became a dangling symlink. When `cat soft.txt` attempts to open the file, the kernel path lookup follows the target pointer to `"hard.txt"`, which no longer exists, returning `ENOENT` (`No such file or directory`). `soft.txt` has a size of exactly **8 bytes** because the target string `"hard.txt"` comprises exactly 8 ASCII bytes. A symlink's size in bytes is strictly the length of its target path string.

4. A new directory has 2 links, and 4 after you create two subdirectories. Explain where each link comes from.

- A newly created directory `d` has a link count of **2**:
  1. Entry `d` in the parent directory (`~/os-lab5/d`).
  2. Self-referential entry `.` inside `d` (`d/.`).
- After creating subdirectories `d/x` and `d/y`, the link count becomes **4**:
  3. Parent directory backlink `..` inside `d/x` (`d/x/..`).
  4. Parent directory backlink `..` inside `d/y` (`d/y/..`).

5. Why does a 2-byte file use 4.0K, and a 1 GB file use 0?

- The filesystem allocates persistent storage in discrete units called **disk blocks** (standard 4096 bytes / 4.0 KB). Even though `one.txt` contains only 2 bytes (`'x'` and `'\n'`), the filesystem must allocate at least one full 4.0 KB block to store it.
- Conversely, `truncate -s 1G big.img` creates a **sparse file**. It updates the inode's logical byte-size metadata to 1,073,741,824 bytes without allocating physical disk blocks. The unwritten span is marked as an empty block hole in the extent tree. Reads over these holes return zeroes synthesized by the kernel without reading physical disk blocks, so physical disk usage is 0 blocks.

Part 3. Permissions

1. File Permissions, Umask, and Directory Traversal

Linux implements POSIX Discretionary Access Control (DAC) using a 9-bit mode string partitioned into three triads: Owner (`u`), Group (`g`), and Others (`o`). Each triad specifies Read (`r`), Write (`w`), and Execute (`x`) rights.

```
$ echo secret > s.txt
$ chmod 600 s.txt ; ls -l s.txt
-rw------- 1 calin calin 7 Nov  2 14:40 s.txt

$ chmod 000 s.txt ; cat s.txt
cat: s.txt: Permission denied

$ chmod u+r s.txt ; cat s.txt
secret

$ umask
0022

$ touch new.txt ; mkdir newdir ; ls -ld new.txt newdir
-rw-r--r-- 1 calin calin    0 Nov  2 14:40 new.txt
drwxr-xr-x 2 calin calin 4096 Nov  2 14:40 newdir
```

Executing the `box` experiment to distinguish directory `r` and `x` bits:

```
$ mkdir box ; echo hi > box/f
$ chmod u=r,go= box ; ls box ; cat box/f
f
cat: box/f: Permission denied

$ chmod u=x,go= box ; ls box ; cat box/f
ls: cannot open directory 'box': Permission denied
hi

$ chmod 755 box
$ ls -l /usr/bin/passwd ; ls -ld /tmp
-rwsr-xr-x 1 root root 80792 Jul 30 13:04 /usr/bin/passwd
drwxrwxrwt 23 root root   560 Nov  2 14:40 /tmp

$ cat /etc/shadow
cat: /etc/shadow: Permission denied

$ ls -l /etc/shadow
-rw------- 1 root root 1428 Oct 20 12:31 /etc/shadow
```

2. Observe Answers for Part 3

1. Write 600, 644, 755 and 700 as rwx strings, and give one real use for each:

- `600` = `-rw-------` : Owner read/write, no access for anyone else. Real use: Confidential user credentials such as private SSH keys (`~/.ssh/id_rsa` , `~/.ssh/id_ed25519`).
- `644` = `-rw-r--r--` : Owner read/write, group and others read-only. Real use: Public text files, system configuration tables (`/etc/passwd` , `/etc/hosts`), and static assets.
- `755` = `-rwxr-xr-x` : Owner full access, group and others read and execute. Real use: Public executable system binaries (`/usr/bin/ls` , `/usr/bin/gcc`) and standard navigable directories.
- `700` = `drwx------` : Owner full access, no access for group or others. Real use: Private user directories requiring complete isolation (`~/.ssh` , `~/.gnupg` , `/root`).

2. With `chmod 000` you own the file but cannot read it. Why does ownership not help? Linux DAC enforces a strict non-falling-through class precedence order: Owner → Group → Others. When a process attempts to open a file, the kernel checks whether the process UID equals the file's owner UID. Because they match, the kernel evaluates **only** the owner permission triad (`---`). Since the read bit is absent, the kernel immediately rejects the request with `EACCES` (`Permission denied`). Ownership confers the right to modify permission attributes (`chmod`), but does not grant implicit read/write access if the corresponding bits are disabled.

3. What is your umask, and how does it explain the permissions of `new.txt` and `newdir` ? The system umask is `0022` (octal `022`).

- For regular files, the default base mode is `0666` (`rw-rw-rw-`). Masking with `~0022` results in `0666 & ~0022 = 0644` (`-rw-r--r--`), stripping write rights from group and others.
- For directories, the default base mode is `0777` (`rwxrwxrwx`). Masking with `~0022` results in `0777 & ~0022 = 0755` (`drwxr-xr-x`), leaving execution/search enabled while restricting directory writes to the owner.

4. Explain the `box` experiment: what does `r` allow on a directory, and what does `x` allow?

- **Directory `r` (Read):** Allows reading the directory table data stream (listing file entries via `opendir` / `readdir` or `ls`). It does not allow traversing into the directory to read file inodes. Thus, with `u=r,go=` , `ls box` succeeds and prints `f` , but `cat box/f` fails with `Permission denied` .

- **Directory `x` (Execute/Search):** Allows directory traversal, resolving paths, and accessing inodes of files within the directory (via `open` or `stat`). It does not allow listing the directory table. Thus, with `u=x,go=`, `ls box` fails with `Permission denied`, but `cat box/f` succeeds and reads `hi`.

5. What do the `s` in `/usr/bin/passwd` and the `t` in `/tmp` mean, and why does `passwd` need the `s`?

- **`s` (SetUID bit):** The SetUID bit on `/usr/bin/passwd` (`-rwsr-xr-x`) causes the process to run with the effective UID of the file owner (`root`) rather than the invoking user.
- **Why `passwd` needs SetUID:** Regular users must be able to change their account passwords. Changing a password requires updating `/etc/shadow`, which is strictly protected with mode `0600` (`-rw-------`) owned by `root`. Without the SetUID bit, unprivileged users would be unable to write to `/etc/shadow`.
- **`t` (Sticky Bit):** The sticky bit on `/tmp` (`drwxrwxrwt`) restricts deletion and renaming in world-writable shared directories. A user can only unlink or rename files that they personally own, preventing users from tampering with each other's temporary files.

Part 4. Disks and the Kernel as Files

1. Storage Disks, Partition Geometry, and `/proc`

The Linux kernel exposes physical block devices, active mount tables, hardware parameters, and per-process memory state as virtual filesystems (`/sys` and `/proc`).

```
$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
zram0       253:0    0     4G  0 disk [SWAP]
nvme0n1     259:0    0 476.9G  0 disk
├─nvme0n1p1 259:1    0     1G  0 part /boot/efi
├─nvme0n1p2 259:2    0    64G  0 part /
└─nvme0n1p3 259:3    0 411.9G  0 part /home

$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p2   64G   38G   22G  64% /
/dev/nvme0n1p3  405G  195G  190G  51% /home
/dev/nvme0n1p1  1.0G   86M  938M   9% /boot/efi
tmpfs           7.5G   46M  7.4G   1% /tmp

$ mount | grep " on / "
/dev/nvme0n1p2 on / type ext4 (rw,relatime)

$ grep -c processor /proc/cpuinfo
8

$ head -3 /proc/meminfo
MemTotal:       15565548 kB
MemFree:         4868992 kB
MemAvailable:   10117156 kB

$ head -5 /proc/self/status
Name: head
Umask:  0022
State:  R (running)
Tgid: 12902
Ngid: 0

$ ls -l /proc/self/fd
total 0
lrwx------ 1 calin calin 64 Nov  2 14:40 0 -> /dev/pts/1
lrwx------ 1 calin calin 64 Nov  2 14:40 1 -> /dev/pts/1
```

```
lrwx------ 1 calin calin 64 Nov  2 14:40 2 -> /dev/pts/1
lr-x------ 1 calin calin 64 Nov  2 14:40 3 -> /proc/12902/fd
```

2. Observe Answers for Part 4

1. Draw your disk: disk name, partitions, which one is mounted at `/` and with what file system type:

```
+-------------------------------------------------------------------------+
| Physical Drive: /dev/nvme0n1 (476.9 GB NVMe Solid State Drive)          |
+-----------------------+---------------------+---------------------------+
| Partition 1 (nvme0n1p1)| Partition 2 (nvme0n1p2) | Partition 3 (nvme0n1p3)   |
| Size: 1.0 GB          | Size: 64.0 GB       | Size: 411.9 GB            |
| Type: vfat (EFI)      | Type: ext4          | Type: ext4 / btrfs        |
| Mount: /boot/efi      | Mount: / (Root)     | Mount: /home              |
+-----------------------+---------------------+---------------------------+
| In-Memory Swap: /dev/zram0 (4.0 GB compressed zram block device)        |
+-------------------------------------------------------------------------+
```

The root partition is `/dev/nvme0n1p2`, mounted at `/` using the **ext4** filesystem (`rw,relatime`).

2. How full is your root file system? The root filesystem `/dev/nvme0n1p2` has a total volume of **64 GB**, with **38 GB** consumed and **22 GB** remaining available, representing **64% capacity utilization**.

3. How many CPU cores and how much RAM does your laptop have, according to `/proc`?

- **CPU Cores: 8 processing cores** (`grep -c processor /proc/cpuinfo`).
- **RAM: 15,565,548 kB (15.2 GB physical RAM)** (`head -3 /proc/meminfo`).

4. `/proc/self/status` showed the name of a program. Which one, and why that one? It displayed **`Name: head`**. **Explanation:** `/proc/self` is a dynamic symbolic link provided by the procfs driver that automatically evaluates to the process ID (`/proc/<PID>`) of the process making the read system call. In the command `head -5 /proc/self/status`, the process executing the `open()` and `read()` system calls on the pseudo-file was `head`. Hence, the status snapshot returned belonged to `head`.

Stretch Goals

1. Magic Numbers

```
$ gzip -c /etc/passwd > notes.gz
$ file notes.gz
notes.gz: gzip compressed data, was "passwd", last modified: Thu Oct  1 09:32:21 2026,
from Unix, original size modulo 2^32 3265

$ xxd notes.gz | head -n 2
00000000: 1f8b 0808 a528 be6a 0003 7061 7373 7764  .....(.j..passwd
00000010: 008d 5659 77e2 3614 7ecf afd0 63fb c0b1  ..VYw.6.~...c...
```

Analysis: The first two bytes of the compressed file are `0x1F 0x8B` (`\x1f\x8b`). Under RFC 1952, these magic header bytes identify a valid GZIP container (`ID1 = 31`, `ID2 = 139`). The `file` utility matches these leading bytes against its signature database (`/usr/share/file/magic`) rather than relying on filename extensions.

2. Deleting an Open File

```
$ truncate -s 10M bigfile
$ exec 3< bigfile
$ rm bigfile
$ ls -l /proc/$$/fd/3
lr-x------ 1 calin calin 64 Nov  2 14:40 /proc/12902/fd/3 -> /home/calin/code/os-labs/
lab5-6/lab5/bigfile (deleted)
$ exec 3<&-
```

Analysis: Opening the file with `exec 3< bigfile` creates an active file descriptor in the shell process table (`ref_count > 0`). When `rm bigfile` executes, the directory entry is unlinked and `nlink` drops to 0. However, the kernel keeps the physical disk blocks and in-memory inode alive because the open file descriptor table retains a reference. Inspecting `/proc/$$/fd/3` reveals the path annotated with `(deleted)`. The storage blocks are only returned to the filesystem's free bitmap when the descriptor is closed via `exec 3<&-` .

Conclusion

Laboratory Work 5 provided practical hands-on verification of Unix filesystem architecture, Discretionary Access Control, and storage management primitives directly on Linux:

1. **Separation of Names and Inodes:** Verified that filenames are strictly directory table records. Hard links provide multiple directory references to the same underlying inode without data duplication, while `rm` merely unlinks directory entries.
2. **Symbolic Links vs. Hard Links:** Demonstrated that symbolic links are distinct files storing target path strings, which become dangling pointers when the target is removed.
3. **Sparse Files vs. Physical Blocks:** Clarified that apparent file size reflects logical metadata, whereas physical storage is allocated in block increments (4.0 KB). Sparse files allow multi-gigabyte logical sizes while consuming zero physical blocks.
4. **Permission Precedence:** Verified that DAC rules evaluate permissions without falling through (Owner → Group → Others), and that directory `x` grants traversal while directory `r` governs entry enumeration.
5. **Kernel as Files:** Demystified how the Linux kernel exposes hardware devices, block storage partitions, and real-time process execution state as virtual files through the `/dev` and `/proc` filesystems.